



Handbook · نسخه ۱.۰ · مهر ۱۴۰۵

راهنمای جامع برنامه‌نویسی عامل‌محور

Spec، Rule، Skill و ارکستراسیون تیم نرم‌افزار با عامل‌های هوش مصنوعی

از نیامندی کسب‌وکار ← Tester ← Developer ← Tech Lead ← Product Owner

Cursor · Claude Code · GitHub Copilot · Codex · Kiro · Spec Kit

github.com/mohammadrezatahmasebi/ai-agentic-meeting

فهرست

1. چرا الان؟ داده‌ها و پارادوکس بهره‌وری
2. مفاهیم پایه: LLM، عامل، Context، Harness
3. چهار artifact: Skill، Rule، Spec و مستندات کسب‌وکار
4. توسعه مبتنی بر Spec – دیدگاه martinfowler.com
5. راهنمای عملی نوشتن Skill و Spec، Rule
6. تیم از A تا Z: نقش‌ها و ارکستراسیون
7. مقایسه ابزارها و ساختار پوشه
8. چطور با عامل صحبت کنیم + ضدالگوها
9. نقشه راه ۳۰/۶۰/۹۰ روزه
10. پرسش‌های پرتکرار (FAQ)
11. واژه‌نامه
12. منابع
13. پیوست: فایل‌های نمونه کامل

نحوه استفاده: این سند همراه اسلایدهای جلسه است. اسلایدها خلاصه‌اند؛ اینجا توضیح کامل، چک‌لیست‌ها و نمونه فایل‌ها آمده است. همه نمونه‌ها در پوشه `samples/` ریپو موجودند و می‌توانید مستقیماً در پروژه‌های NET کپی کنید.

۱. چرا الان؟ داده‌ها و پارادوکس بهره‌وری

ابزارهای کدنویسی با هوش مصنوعی در چند سال اخیر چهار نسل را پشت سر گذاشته‌اند: تکمیل خودکار (یک خط یا یک تابع)، چت‌بات (پرسش و پاسخ و کپی‌پیست)، عامل (Agent)؛ خودش فایل می‌خواند، ویرایش می‌کند، ترمینال و تست اجرا می‌کند) و اکنون تیم عامل‌ها (یک هماهنگ‌کننده و چند عامل متخصص).

دو داده متضاد

منبع	یافته	روش
DORA 2025 (Google Cloud)	۹۰٪ توسعه‌دهندگان از AI استفاده می‌کنند؛ بیش از ۸۰٪ احساس بهره‌وری بیشتر دارند. AI توان تحویل (throughput) را بالا برده ولی پایداری تحویل را کاهش داده. ۳۰٪ اعتماد کم یا هیچ به کد AI دارند.	نظرسنجی ~۵۰۰۰ نفر + داده کیفی
METR (ژوئیه ۲۰۲۵)	توسعه‌دهندگان باتجربه در ریپوهای بزرگ خودشان با ۱۹٪ AI کندتر شدند، در حالی که باور داشتند ۲۰٪ سریع‌ترند.	آزمایش کنترل‌شده تصادفی، ۲۴۶ تسک واقعی

«نقش اصلی AI در توسعه نرم‌افزار، نقش یک تقویت‌کننده است: نقاط قوت سازمان‌های پرمالکد و اختلال‌های سازمان‌های ناکارآمد را بزرگ‌تر می‌کند.» — DORA 2025

درس برای تیم ما: خرید ابزار به تنهایی بهره‌وری نمی‌آورد. آنچه تفاوت ایجاد می‌کند فرایند است: context درست (Spec، Rule، Skill)، حلقه بازخورد قطعی (تست، build) و بازبینی. احساس ما از سریع‌تر شدن قابل اعتماد نیست؛ باید اندازه بگیریم.

۲. مفاهیم پایه: LLM، عامل، Context، Harness

۲.۱ تفاوت LLM، دستیار کدنویسی و عامل

تعریف	LLM	دستیار کدنویسی AI	عامل کدنویسی
مدل زبانی: پیش‌بینی توکن بعدی	مدل + رابط کاربری در IDE	مدل + ابزارها + حلقه تصمیم‌گیری	
مثال	Claude, GPT, Gemini	Copilot completions, چت IDE	Cursor Agent, Claude Code, Codex, Kiro, Copilot Agent
دسترسی	فقط متن ورودی	فایل باز یا انتخاب‌شده	کل ریپو، ترمینال، تست، مرورگر، MCP
اجراکننده	شما	شما (قبول/کپی)	خود عامل؛ شما هدایت و بازبینی
بهترین کاربرد	تحلیل، ایده، توضیح	تکمیل سریع، سؤال موضعی	فیچر چندفایلی، رفع باگ، refactor، تست‌نویسی

جمله کلیدی: **Agent = Model + Harness** (Birgitta Böckeler, martinoflower.com). یعنی «کدام مدل بهتر است» فقط نیمی از سؤال است؛ نیم دیگر این است که دور مدل چه ساخته‌اید.

۲.۲ حلقه عامل

عامل در یک حلقه کار می‌کند: **هدف** را دریافت می‌کند، **برنامه** می‌ریزد، **ابزار** اجرا می‌کند (خواندن/جستجو/ویرایش فایل، ترمینال، تست، مرورگر)، **نتیجه** را مشاهده می‌کند و تا رسیدن به هدف تکرار می‌کند.

هشدار: اگر معیار موفقیت قطعی (تست، lint، build) ندهید، عامل خودش به کار خودش نمره می‌دهد. ارزیابی خود مدل از «تمام شد» کم‌اعتمادترین سیگنال در حلقه است؛ نتیجه تست قابل‌اعتمادترین.

۲.۳ مهندسی زمینه (Context Engineering)

«Context engineering یعنی گزینش آنچه مدل می‌بیند، تا نتیجه بهتری بگیرد.» – Bharani
Subramaniam، در مقاله «Context Engineering for Coding Agents» نوشته Birgitta
Böckeler، martinowler.com، فوریه ۲۰۲۶

پنجره context محدود است و هر چیز بی‌ربط کیفیت را پایین می‌آورد. ابزارهای پیکربندی context در عامل‌ها:

- **Rules** دستورالعمل‌های دائمی پروژه (همیشه یا با الگوی مسیر بارگذاری می‌شوند)
- **Skills** رویه‌هایی که فقط هنگام نیاز بارگذاری می‌شوند
- **Specs** قرارداد فیچر در حال ساخت
- **MCP** اتصال به سیستم‌های بیرونی (Jira، GitLab، دیتابیس، مستندات، مرورگر)
- **Subagents** عامل‌های کمکی با context جداگانه
- **خود کدبیس** – Böckeler تأکید می‌کند که طراحی کد هم بخشی از context است؛ کد تمیز و یکدست، خروجی بهتر عامل.

۲.۴ مهار (Harness): راهنماها و حسگرها

در مقاله «Harness engineering for coding agent users» در Harness، martinowler.com هر چیزی در عامل است جز خود مدل. Harness بیرونی که ما می‌سازیم دو هدف دارد: بالا بردن احتمال درست انجام دادن کار در بار اول، و ایجاد حلقه بازخوردی که تا حد ممکن خطاها را قبل از رسیدن به انسان اصلاح کند.

حسگرها (Sensors) – بعد از عمل

- Build / Compiler
- Unit و Integration Test
- Linter و Formatter
- تست‌های معماری (مثلاً NetArchTest)
- عامل بازبین read-only

راهنماها (Guides) – قبل از عمل

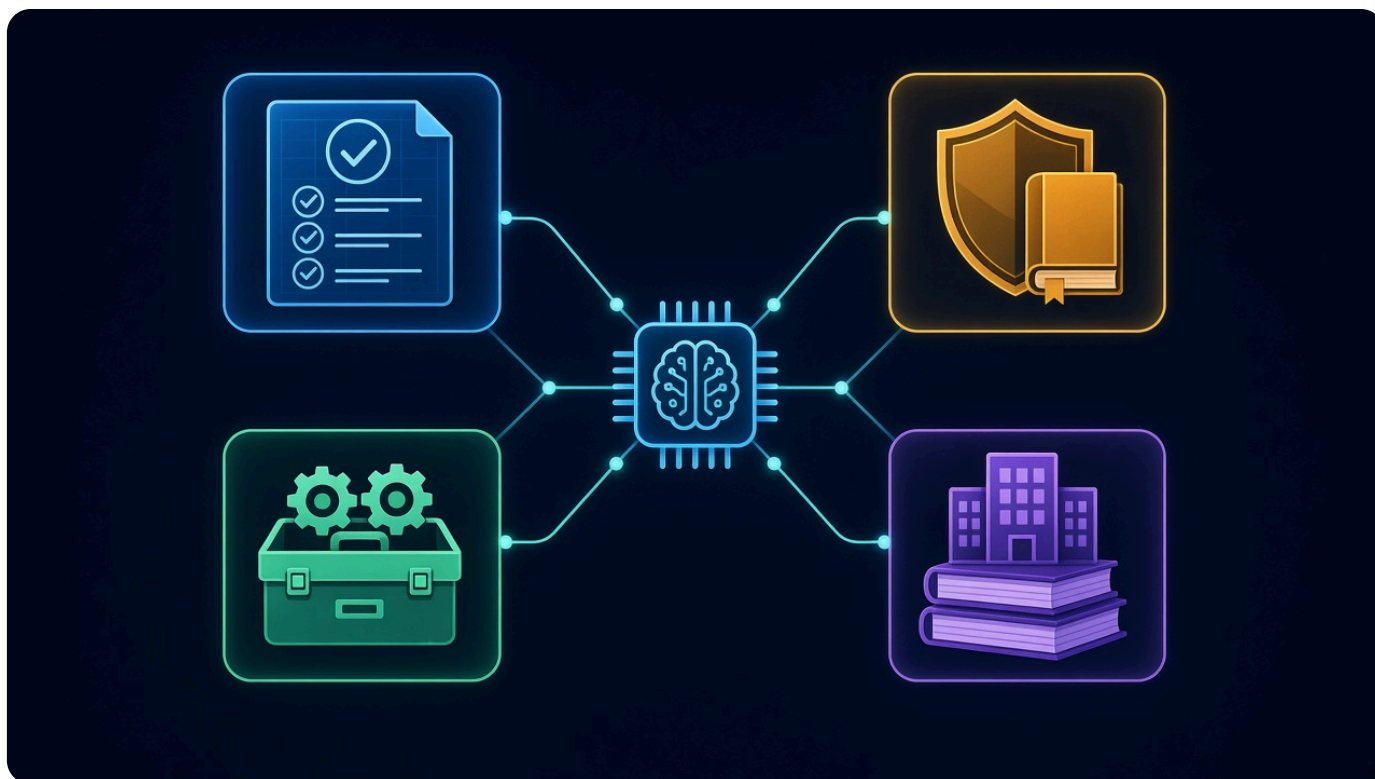
- Rules و AGENTS.md
- Spec و معیار پذیرش
- Skills
- ADR و اسناد معماری
- نمونه کد خوب در ریپو

۲.۵ Hook و MCP، Subagent

- **MCP (Model Context Protocol)**: پروتکل استاندارد برای اتصال عامل به ابزارها و داده‌های بیرونی.

- **Subagent:** عامل کمکی با context و ابزارهای محدود خودش که فقط خلاصه نتیجه را برمی‌گرداند. برای تحقیق در کدبیس (تا context اصلی آلوده نشود) و بازبینی بی‌طرف عالی است.
- **Hook:** اسکرپت قطعی که در رویدادهای چرخه عامل اجرا می‌شود (قبل/بعد از ابزار، پایان کار). Anthropic: «برخلاف دستورالعمل‌های CLAUDE.md که توصیه‌ای هستند، Hook ها قطعی‌اند و تضمین می‌کنند کار انجام شود.»

۳. چهار artifact: Spec، Rule، Skill و مستندات کسب‌وکار



مهم‌ترین مفهوم این راهنما: این چهار مورد چیزهای متفاوتی هستند و قاطی کردن آن‌ها رایج‌ترین اشتباه تیم‌هاست.

مالک	کی بارگذاری می‌شود	عمر	چیست	Artifact
PO + Tech Lead	فقط هنگام کار روی همان فیچر	به ازای فیچر	توصیف این فیچر/تغییر: هدف، داستان کاربر، معیار پذیرش، Non-goals، لبه‌ها	Spec
Tech Lead	همیشه / با glob / به درخواست عامل	طولانی	سیاست دائمی پروژه: معماری، نام‌گذاری، تست، امنیت، دستورهای build	Rule
Tech Lead + ارشدها	در صورت نیاز، وقتی کار با description جور شود	طولانی	رویه قابل‌استفاده مجدد: چطور یک نوع کار را انجام دهیم	Skill
PO / تحلیلگر	خودکار نه؛ از Spec لینک یا Skill references یک در	طولانی	دانش دامنه: واژه‌نامه، روایت فرایند، BRD، قوانین کسب‌وکار	Business Doc

جمله‌های حفظ‌کردنی:

Spec = این بار چه بسازیم · **Rule** = اینجا همیشه چطور کار می‌کنیم · **Skill** = این نوع کار را چطور انجام دهیم · **Doc** = کسب‌وکار چه معنایی دارد

۳.۱ Spec چیست؟

Spec قرارداد بین کسب‌وکار، تیم فنی و عامل است – قبل از نوشتن کد. در محل `specs/NNN-feature/spec.md` قرار می‌گیرد، در git نسخه‌بندی می‌شود و در PR بازبینی می‌شود. مهم‌ترین بخش آن معیار پذیرش قابل تست است؛ همین معیارها بعداً تست می‌شوند و حسگر عامل را تشکیل می‌دهند.

۳.۲ Rule چیست؟

Rule دستورالعمل دائمی است که عامل در تقریباً هر ویرایش مرتبط رعایت می‌کند. چون Rule ها همیشه (یا با هر فایل منطبق) در context بارگذاری می‌شوند، هر کلمه اضافه هزینه دائمی دارد. پس کوتاه، دستوری و دائمی بنویسید. محل‌ها: `AGENTS.md` ، `.cursor/rules/*.mdc` ، `CLAUDE.md` ، `.github/copilot-instructions.md`.

۳.۳ Skill چیست؟

Skill یک پوشه با فایل `SKILL.md` است (استاندارد باز `agentskills.io` که اصالتاً توسط Anthropic ساخته شد و اکنون Claude Code، GitHub Copilot، Cursor و Codex پشتیبانی می‌کنند). فایل شامل frontmatter با `name` و `description` و بدنه دستورالعمل است و می‌تواند پوشه‌های اختیاری `scripts/` ، `references/` و `assets/` داشته باشد.

بارگذاری تدریجی (Progressive Disclosure)

- کشف: در شروع فقط `name` و `description` همه Skill ها (۱۰۰~ توکن هرکدام)
 - فعال‌سازی: وقتی کار با `description` جور شد، کل `SKILL.md` (توصیه: کمتر از ۵۰۰۰ توکن)
 - اجرا: فایل‌های `references/scripts` فقط در صورت نیاز
- به همین دلیل می‌توانید ده‌ها Skill داشته باشید بدون اینکه context شلوغ شود. فیلد `description` تمام بار تشخیص «کی فعال شود» را به دوش می‌کشد.

۳.۴ مستندات کسب‌وکار – و چرا «قانون کسب‌وکار» $\neq$ Rule عامل

این پرتکرارترین سؤال است: «قوانین کسب‌وکار ما Rule هستند یا Skill یا سند؟»

مثال: «اقساط معوق بیش از ۳۰ روز باید پیگیری شوند.»

- ✓ در مستند دامنه (docs/domain/installments.md) – حقیقت برای انسان‌ها
- ✓ در Spec فیچر مربوط، به شکل معیار پذیرش Given/When/Then
- ✓ اختیاری: در references/ یک Skill، اگر عامل‌ها مکرر به واژه‌نامه دامنه نیاز دارند
- ✗ در cursor/rules یا AGENTS.md – مگر اینکه قرارداد دائمی کدنویسی باشد، مثل «فیلدهای پولی همیشه decimal»

۳.۵ درخت تصمیم

آیا این متن فقط برای یک فیچر/تغییر فعلی است؟

└ بله → SPEC (specs/.../spec.md)

└ خیر

آیا قرارداد دائمی کدنویسی / معماری / امنیت است؟

└ بله → RULE (AGENTS.md یا cursor/rules/*.mdc)

└ خیر

آیا رویه تکراری «چطور انجام بده» است؟

└ بله → SKILL (.agents/skills/.../SKILL.md)

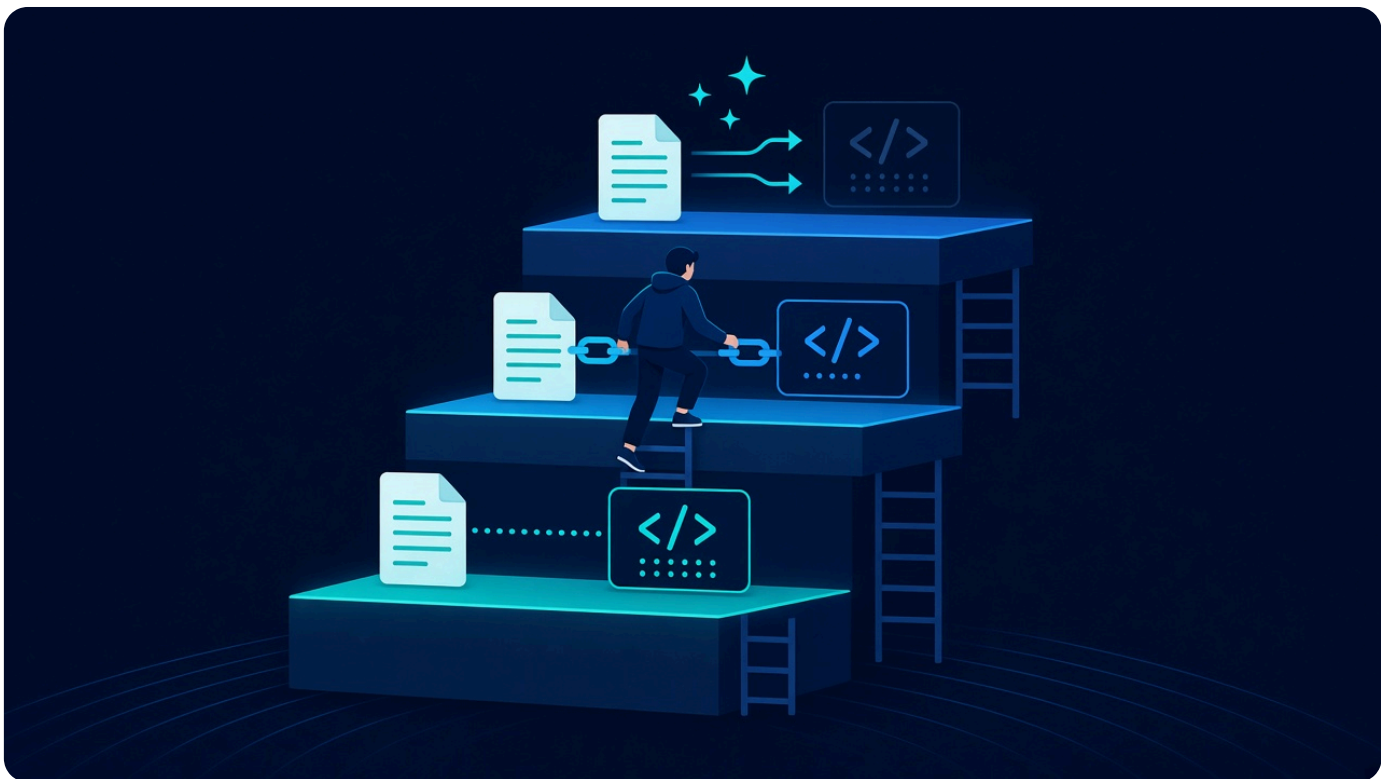
└ خیر → BUSINESS DOC (docs/domain/) و از Spec بده لینک

جمله	کجا؟
اقساط معوق بیش از ۳۰ روز follow-up شوند	Doc + Spec (معیار پذیرش)
Domain نباید به Infrastructure وابسته باشد	Rule
مراحل افزودن فیچر CQRS با MediatR	Skill
در این اسپرینت endpoint پیگیری اقساط بساز	Spec
واژه‌نامه: قسط، بیمه‌نامه، فروشنده، معوق	Doc
رمز و connection string هرگز در کد	Rule (+ Hook برای اسکن secret)

۳.۶ Skills در برابر Rules

Skills	Rules	
آموزش قابلیت یا گردش کار تخصصی	استاندارد و قرارداد کدنویسی	هدف
در صورت نیاز، تدریجی	همیشه یا با glob (کل متن)	بارگذاری
دستورالعمل + اسکریپت + مثال + منابع	فقط دستورالعمل	محتوا
می تواند مفصل باشد	کوتاه	طول
استاندارد باز واحد	فرمت متفاوت به ازای هر ابزار	قابلیت حمل
Skill همیشه لازم و کوتاه است ← Rule کن	Rule شبیه آموزش گام به گام شد ← Skill کن	نشانه جابه جایی

۴. توسعه مبتنی بر Spec – دیدگاه martinowler.com



مرجع اصلی این فصل مقاله «[Understanding Spec-Driven-Development: Kiro, spec-kit, and Tessel](#)» نوشته **Birgitta Böckeler** (Distinguished Engineer در Thoughtworks) در سایت Martin Fowler است، به همراه پادکست Thoughtworks با حضور او و Laura Tacho.

۴.۱ مسئله: Vibe Coding

«Vibe coding» یعنی پرامپت ← کد ← امید. برای نمونه‌سازی سریع خوب است، اما در محصول مشکل اصلی هم‌راستایی است: عامل ممکن است چیز اشتباهی را کاملاً درست بسازد. Laura Tacho (DX) در پادکست می‌گوید طبق داده‌هایشان مشکل پایدار تیم‌ها کمبود هم‌راستایی است و SDD تلاشی برای حل بخشی از همین است.

۴.۲ تعریف Böckeler

SDD یک گردش کار است برای ایجاد Spec خوش‌فکرشده در فرایند برنامه‌ریزی، قبل از پیاده‌سازی، که سپس به عامل تحویل داده می‌شود. ارزش کلیدی آن هم‌راستایی و همگام نگه داشتن Spec با کد است.

نتیجه‌گیری اصلی او: «مثل هر مفهوم نوظهور، اصطلاحات هنوز بسیار سیال است و spec-driven می‌تواند معانی مختلفی داشته باشد. حتی درک مشترکی از اینکه Spec چیست و چه تفاوتی با instruction file یا پرامپت دارد وجود ندارد.»

۴.۳ سه سطح SDD

سطح	تعریف Böckeler	پیشنهاد برای ما
Spec-first	Spec خوش‌فکرشده اول نوشته و برای همان تسک در گردش‌کار AI استفاده می‌شود.	نقطه شروع برای همه فیچرها
Spec-anchored	Spec بعد از اتمام تسک نگه داشته می‌شود تا برای تکامل و نگهداری همان فیچر استفاده شود.	برای ماژول‌های اصلی و پرتغییر (اقساط، بیمه‌نامه، پرداخت)
Spec-as-source	Spec فایل اصلی است؛ فقط Spec توسط انسان ویرایش می‌شود و انسان هرگز کد را لمس نمی‌کند.	فعلاً فقط آزمایش؛ ابزارها در مرحله beta

۴.۴ سه ابزار از نگاه Böckeler

- **Kiro (AWS)**: ساده‌ترین و سبک‌ترین؛ جریان Tasks ← Design ← Requirements. عمدتاً spec-first.
- **Spec Kit (GitHub)**: به‌صورت CLI توزیع می‌شود و ساختار workspace را برای طیف وسیعی از دستیارها می‌سازد؛ تعامل با slash command. «چون همه artifact ها در workspace شما قرار می‌گیرند، این سفارشی‌پذیرترین ابزار از این سه است.» با این حال در عمل spec-first است.
- **Tessl**: تنها ابزاری که صریحاً به سمت spec-anchored و حتی spec-as-source می‌رود. کد تولیدی با کامنت `// GENERATED FROM SPEC – DO NOT EDIT` علامت می‌خورد؛ فعللاً نگاشت ۱:۱ بین Spec و فایل. هنوز beta است.

۴.۵ جریان Spec Kit

فرمان	خروجی	معادل در مدل ما	دروازه انسانی
<code>/constitution</code>	اصول دائمی پروژه	Rule	Tech Lead
<code>/specify</code>	<code>spec.md</code> – چه و چرا	Spec	PO + Tech Lead تأیید
<code>/plan</code>	<code>plan.md</code> – چگونه (فنی)	Plan / ADR	Tech Lead تأیید
<code>/tasks</code>	<code>tasks.md</code> – تکه‌های کوچک	Task list	–
<code>/implement</code>	کد + تست	با Skill ها	بازبینی PR

۴.۶ SDD چطور به تیم ما کمک می‌کند

۱. هم‌راستایی زودهنگام: PO و Tech Lead قبل از مصرف توکن و وقت توسعه توافق می‌کنند.
۲. دروازه کیفیت: معیار پذیرش ← تست ← حسگر خودکار عامل.
۳. کاهش دوباره‌کاری: کمتر «عامل چیز اشتباهی ساخت».
۴. تحویل صریح بین نقش‌ها: Tasks ← Plan ← Spec ← BRD ← کد ← تست.
۵. بازبینی بهتر: بازبین در برابر Spec چک می‌کند، نه حافظه یا حدس.
۶. رشد جونیورها: Spec بریف کار است و Skill روش انجام.
۷. حافظه سازمانی: چرا یک فیچر این‌طور ساخته شد، در ریپو می‌ماند – نه در تاریخچه چت.

۴.۷ نقدها و هشدارها

- اصطلاحات سیال: «Spec» برای افراد مختلف معانی متفاوت دارد؛ در تیم تعریف مشترک بگذارید (همین فصل ۳).
- آبشار برای عامل‌ها: برخی کاربران Spec Kit را «waterfall برای عامل‌ها» و بیش از حد سنگین توصیف کرده‌اند. Spec را سبک و اندازه یک PR نگه دارید.
- رانش (Drift): در spec-first اگر Spec به‌روز نشود، بعداً گمراه‌کننده است. برای فیچرهای مهم spec-anchored.
- Spec جای فکر کردن را نمی‌گیرد: فرم Spec به‌تنهایی پرامپت بهتر نمی‌سازد؛ تنظیمات خاص پروژه (ابزار، محیط) را هم باید صریح بنویسید.

۴.۸ مقالات مرتبط در martinfowler.com

- [Context Engineering for Coding Agents](#) — Rules, Skills, MCP و context
- [Harness engineering for coding agent users](#) — راهنماها و حسگرها
- [Humans and Agents in Software Engineering Loops](#) — Kief Morris، «روی حلقه»

۵. راهنمای عملی نوشتن Spec، Rule و Skill

۵.۱ نوشتن Spec

قالب حداقلی

1. هدف – یک پاراگراف درباره نتیجه
2. داستان‌های کاربر – «به‌عنوان ... می‌خواهم ... تا ...»
3. معیار پذیرش – Given/When/Then یا چک‌لیست؛ قابل تست
4. خارج از محدوده – جلوی کار اضافه عامل را می‌گیرد
5. قوانین دامنه مرتبط – نقل کوتاه یا لینک به `docs/domain`
6. لبه‌ها و خطاها
7. وابستگی‌ها – API، جدول، رویداد
8. Definition of Done – تست، مستند، observability

چک‌لیست کیفیت

- هر معیار پذیرش با یک تست یا دمو قابل اثبات است
 - معماری مفصل اینجا نیست (برود در `plan.md` یا `ADR`)
 - اندازه یک PR یا اپیک کوتاه است
 - واژه‌ها با واژه‌نامه کسب‌وکار یکی است
 - Non-goals صریح است
 - بعد از merge، برای فیچر مهم نگه داشته و به‌روز می‌شود
- نمونه کامل: پیوست، فایل `specs/001-installment-followup/spec.md`.

۵.۲ نوشتن Rule

چه چیزی Rule باشد

- مرزهای معماری (Clean Architecture)
- قراردادهای نام‌گذاری و پوشه
- حداقل‌های تست

- لیست «هرگز» امنیتی
- دستورهای build/test که عامل باید اجرا کند

ویژگی‌های Rule خوب و بد

✗ بد

- مقاله یا آموزش بلند (← Skill)
- دستور مخصوص یک فیچر (← Spec)
- کپی (← docs) BRD
- تکرار محتوای AGENTS.md در هر فایل ابزار

✓ خوب

- کوتاه و دستوری (Do / Don't)
- دائمی
- با globs محدود
- چند فایل کوچک به جای یک فایل غول
- مالک مشخص، review مثل کد

فرمت .mdc در Cursor

```
---
description: Clean Architecture boundaries for .NET projects
globs: "**/*.{cs,csproj}"
alwaysApply: false
---
# .NET Clean Architecture
## Do
- Depend inward (Host → Application → Domain).
## Don't
- Don't reference Infrastructure from Domain.
```

Cursor تنها فرمتی است که فعال‌سازی جداگانه برای هر Rule دارد: همیشه (`alwaysApply: true`)، با الگوی فایل (`globs`)، به تشخیص عامل (با `description`) یا دستی. فایل قدیمی `.cursorrules` منسوخ است.

۵.۳ نوشتن Skill

ساختار

```
.agents/skills/dotnet-cqrs-feature/
SKILL.md           # required: frontmatter + instructions
references/        # optional: deep docs, loaded on demand
scripts/           # optional: helper scripts
assets/            # optional: templates
```

قواعد frontmatter

- **name** : حداکثر ۶۴ کاراکتر، حروف کوچک، عدد و خط تیره؛ هم‌نام پوشه
- **description** : حداکثر ۱۰۲۴ کاراکتر؛ هم «چه می‌کند» هم «کی استفاده شود»
- اختیاری: **license** ، **compatibility** ، **metadata** ، **allowed-tools** ؛ در Cursor همچنین **paths** برای محدود کردن به فایل‌ها

بدنه

- مراحل شماره‌دار با دستورهای دقیق
- پیش‌نیازها (مثلاً «اول Spec را بخوان»)
- چک‌لیست خروجی
- بخش «هرگز» برای ریسک‌ها
- زیر ۵۰۰۰ توکن؛ جزئیات اضافه در [references/](#)

چک‌لیست

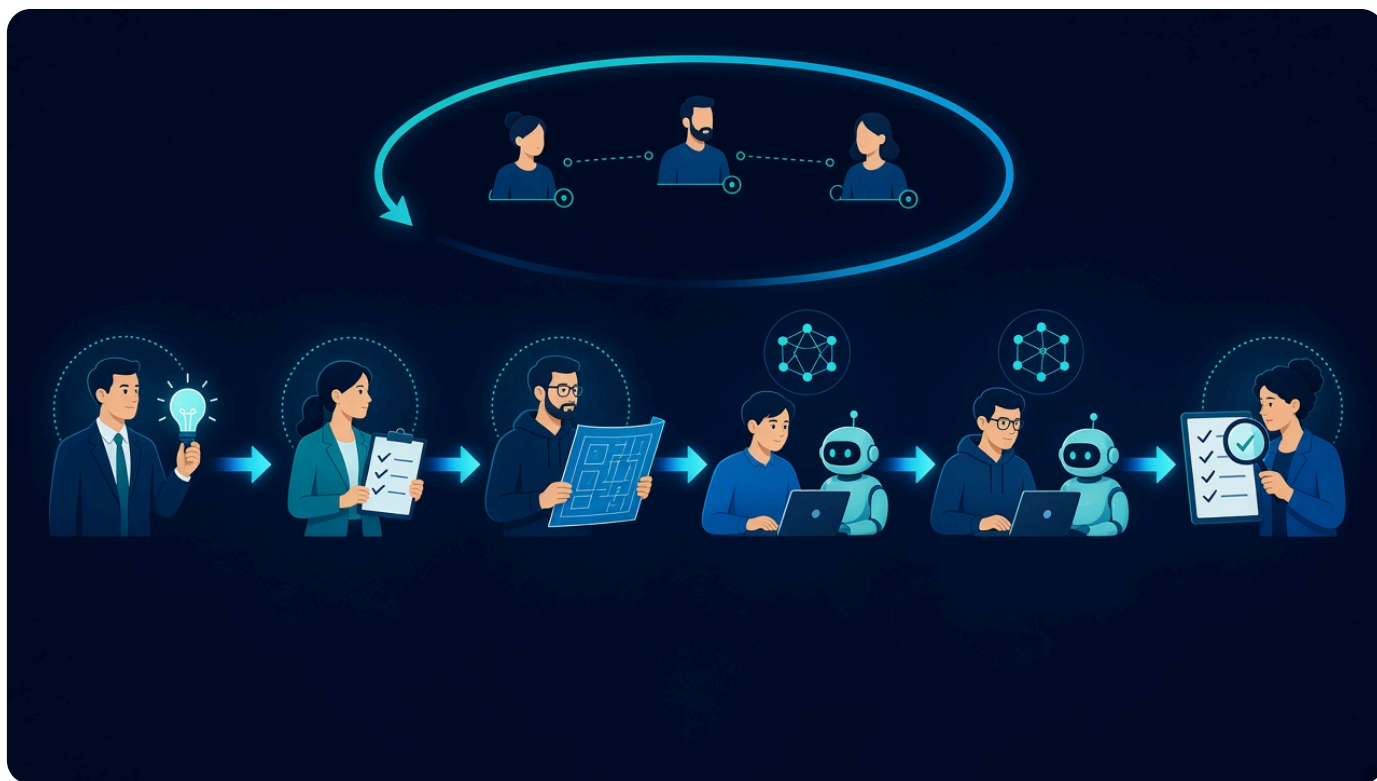
- ☐ description شامل trigger واضح است («... Use when»)
- ☐ رویه تکراری است، نه فیچر یک‌باره
- ☐ با یک کار واقعی تیم تست شده است
- ☐ به Spec و Rule ارجاع می‌دهد، تکرارشان نمی‌کند

۵.۴ ساختار مستندات کسب‌وکار در ریپو

```
docs/
  domain/glossary.md      # business language
  domain/installments.md  # business process narrative
  adr/                    # architecture decisions
  specs/001-.../spec.md   # feature specs (agent + human)
  AGENTS.md + .cursor/rules/ # standing agent rules
  .agents/skills/         # procedures
```

جریان: تحلیلگر/PO مستندات [docs/domain](#) را نگه می‌دارد ← برای هر فیچر معیارهای پذیرش استخراج و در [spec.md](#) نوشته می‌شود ← عامل با Skills + Rules + Spec کار می‌کند.

۶. تیم از A تا Z: نقش‌ها و ارکستراسیون



۶.۱ جریان کامل

نقش	ورودی	کار با عامل	خروجی
کسب و کار	مسئله / فرصت	خلاصه سازی جلسات، پیش نویس BRD	BRD ، docs/domain
Product Owner	BRD	تبدیل به داستان کاربر، پیشنهاد معیار پذیرش و لبه ها، ساخت تیکت Jira با MCP، کشف ابهام	spec.md تأیید شده
Tech Lead	Spec	Plan Mode برای طراحی، نگهداری Rules/Skills، تقسیم Tasks، طراحی Harness	plan.md ، tasks.md ، Rules، Skills، ADR
Developer	Plan + Tasks	Explore ← Plan ← Implement ← Verify ← Review	Pull Request
Tester / QA	Spec + PR	تبدیل AC به test case، تولید تست integration/E2E، یافتن لبه های پوشش نیافته، تست UI با عامل مرورگر	گزارش PASS/FAIL، شکاف های Spec

۶.۲ هر نقش چه سؤالی از عامل بپرسد

Tech Lead

- «در Plan Mode برای spec.md یک plan با فایل ها، ترتیب و ریسک ها بده.»
- «الگوهای تکراری PR های اخیر را پیدا کن و یک Skill پیشنهاد بده.»
- «کجای کد Rule معماری را نقض کرده؟»

Product Owner

- «از این BRD داستان های کاربر و معیار پذیرش Given/When/Then استخراج کن.»
- «چه لبه ها و حالت خطایی در این Spec جا افتاده؟»
- «تناقض های این Spec با docs/domain/installments.md را پیدا کن.»

Tester

- «برای هر معیار پذیرش spec.md یک test case و تست integration بنویس.»
- «چه سناریوهایی در Spec نیامده ولی در کد ممکن است؟»
- «این فرم را در مرورگر تست کن و گزارش بده.»

Developer

- «با subagent بررسی کن InstallmentService فعلی چطور کار می‌کند.»
- «Task T2 را با Skill dotnet-cqrs-feature پیاده کن و dotnet test را سبز کن.»
- «با یک subagent بازبینی‌ی‌طرف diff را بررسی کن.»

۶.۳ الگوهای ارکستراسیون

- **Orchestrator + متخصص‌ها:** یک عامل هماهنگ‌کننده که خودش کد نمی‌نویسد، کار را بین Explorer، Implementer، Tester، Reviewer (read-only) و (read-only) تقسیم می‌کند.
- **Context جدا:** هر subagent context و ابزار محدود خودش را دارد و فقط خلاصه برمی‌گرداند.
- **کار موازی در git worktree:** هر Implementer در یک worktree/branch جدا.
- **دروازه‌های انسانی:** تأیید Plan و تأیید Merge.
- **سقف تکرار:** مثلاً حداکثر ۳ دور اصلاح برای هر تسک، سپس ارجاع به انسان.
- **PR = عامل PR = انسان:** همان branch protection، CI و بازبینی.

۶.۴ انسان درون حلقه یا روی حلقه؟

Kief Morris در «Humans and Agents in Software Engineering Loops» سه حالت را مقایسه می‌کند:

حالت	توضیح	نتیجه
Vibe coding	انسان بیرون؛ هر چه عامل ساخت قبول	مناسب نمونه‌سازی
In the loop	انسان هر خط را بازبینی و اصلاح می‌کند	گلوگاه
On the loop	انسان Harness (Spec، تست، pipeline، Skill، Rule) را می‌سازد و بهبود می‌دهد	مقیاس‌پذیر

تفاوت «درون» و «روی» حلقه وقتی دیده می‌شود که از خروجی عامل راضی نیستیم. روش «درون حلقه»: اصلاح همان خروجی. روش «روی حلقه»: اصلاح Harness ای که آن خروجی را تولید کرد تا نتیجه دلخواه را بدهد. — Kief Morris

۶.۵ دروازه «انجام شد» و سطح خودمختاری

دروازه تأیید پیشنهادی برای هر تغییر عامل:

1. تست‌های جدید/مرتبط پاس
2. کل suite پاس (بدون regression)
3. Build و lint تمیز
4. معیارهای پذیرش Spec پوشش داده شده
5. برای تغییرات پرریسک: تأیید انسانی

خودمختاری یک تنظیم سراسری نیست؛ ویژگی نوع کار و قدرت تأیید آن است:

نوع کار	خودمختاری پیشنهادی
مستندات، تست‌نویسی مازول‌های پوشش‌دار	بالا
ارتقای وابستگی با پوشش تست کامل	بالا
فیچر معمولی با Spec	متوسط – تأیید Plan قبل از اجرا
احراز هویت، پرداخت، صدور بیمه‌نامه	پایین – فقط پیشنهاد، تأیید انسانی
Migration داده	پایین + تأیید صریح

۷. مقایسه ابزارها و ساختار پوشه

۷.۱ ماتریس ابزارها (۲۰۲۶)

ابزار	رابط	مدل‌ها	نقطه قوت	مناسب برای
Cursor	IDE کامل + عامل پس‌زمینه	چند ارائه‌دهنده	ویرایش تعاملی، Rules با Skills، glob، subagent	کار روزانه در IDE
Claude Code	ترمینال + IDE	Anthropic	استدلال عمیق، refactor بزرگ، Hook، Skill، subagent	کارهای چندفایلی سخت
GitHub Copilot	افزونه + IDE GitHub + CLI	چند ارائه‌دهنده	code review، Issue→PR، یکپارچگی سازمانی	سازمان‌های GitHub-محور
OpenAI Codex	ابزار + ابر	OpenAI	تسک ناهمگام در sandbox ابری	کار موازی «بفرست و فراموش کن»
Kiro	IDE	چند ارائه‌دهنده	SDD داخلی (Requirements/Design/Tasks)	تیم‌های AWS، شروع SDD
Spec Kit	CLI روی ابزار شما	—	قالب SDD برای ~۳۰ عامل	افزودن SDD به ابزار فعلی

قیمت‌ها در ۲۰۲۶ عموماً از ۱۰ تا ۲۰ دلار در ماه شروع می‌شوند، ولی مصرف عامل با credit یا پنجره مصرف محاسبه می‌شود؛ قیمت صندلی کف هزینه است. بسیاری از تیم‌ها دو ابزار را ترکیب می‌کنند: یکی برای ویرایش روزانه و یکی برای کار سنگین واگذارشده.

۷.۲ هر ابزار Rule و Skill را از کجا می‌خواند

ابزار	Rules / Instructions	Skills
Cursor	ریشه <code>CLAUDE.md</code> ، <code>AGENTS.md</code> ، <code>.cursor/rules/*.mdc</code>	<code>.agents/skills</code> ، <code>.cursor/skills</code>
Claude Code	<code>CLAUDE.md</code> (با <code>@AGENTS.md</code> import)	<code>.claude/skills</code> : subagent در <code>.claude/agents</code>
GitHub Copilot	<code>.github/copilot-instructions.md</code> ، <code>AGENTS.md</code> ، <code>.github/instructions/*.instructions.md</code>	<code>.github/skills</code> ، <code>.agents/skills</code> ، <code>.claude/skills</code>
Codex	<code>AGENTS.md</code> (تودرتو؛ نزدیک‌ترین برنده)	<code>.agents/skills</code>
Kiro	<code>.kiro/steering/*.md</code>	—

AGENTS.md استاندارد بی‌طرف است: تحت مدیریت بنیاد Linux از دسامبر ۲۰۲۵، در بیش از ۶۰ هزار ریپو، و توسط بیش از ۳۰ ابزار خوانده می‌شود. قاعده: نزدیک‌ترین `AGENTS.md` به فایل ویرایش‌شده برنده است و دستور صریح کاربر در چت بر همه مقدم است.

۷.۳ ساختار پوشه پیشنهادی

```
repo/
├── AGENTS.md           # single source of truth (rules)
├── CLAUDE.md           # "@AGENTS.md" + Claude-only notes
├── .github/copilot-instructions.md  # short, points to AGENTS.md
├── .cursor/rules/
│   ├── dotnet-clean-architecture.mdc
│   └── testing.mdc
├── .agents/skills/
│   ├── dotnet-cqrs-feature/SKILL.md
│   └── ef-migration/SKILL.md
├── .claude/agents/code-reviewer.md
├── specs/001-installment-followup/{spec,plan,tasks}.md
├── docs/domain/ docs/adr/
└── src/ tests/
```

۷.۴ کدام ابزار برای کدام کار

پیشنهاد	کار
چت LLM یا Ask mode در IDE	سؤال سریع، توضیح کد
Cursor Agent / Copilot Agent mode	ویرایش روزانه، فیچر کوچک
Plan Mode + subagent یا Claude Code یا Cursor	Refactor بزرگ، باگ پیچیده
Codex cloud، Cursor background agents، Copilot coding agent	کار موازی ناهمگام
Kiro یا Spec Kit	فیچر با ذی‌نفعان متعدد
read-only Copilot code review، Bugbot، subagent	بازبینی PR

مهم‌تر از انتخاب ابزار: Harness شما (AGENTS.md، Specs، Skills، تست‌ها) بین ابزارها **قابل انتقال** است و شما را به یک فروشنده وابسته نمی‌کند.

۸. چطور با عامل صحبت کنیم + ضدالگوها

۸.۱ آناتومی پرامپت خوب

1. هدف: چه نتیجه‌ای؟ (نه فقط «درستش کن»)
2. Context: Spec، فایل‌های مرتبط، کد مشابه موجود، تیکت
3. محدودیت‌ها: چه چیزی تغییر نکند، Non-goals، الگوی موجود
4. تأیید: چطور بفهمیم درست است؟ تست، build، معیار پذیرش
5. خروجی: Plan؟ PR؟ خلاصه؟ قالب مشخص
6. حالت: اول Plan Mode، بعد اجرا

قاعده Anthropic: اگر diff را می‌توانید در یک جمله توصیف کنید، Plan لازم نیست؛ وگرنه اول Explore و Plan.

۸.۲ مثال

✗ بد: «یه API برای پیگیری اقساط بنویس.»

✓ خوب:

- فیچر `specs/001-installment-followup/spec.md` را پیاده کن.
۱. اول با subagent بررسی کن InstallmentService فعلی چطور کار می‌کند.
 ۲. در Plan Mode فایل‌ها و ترتیب را پیشنهاد بده و صبر کن تا تأیید کنم.
 ۳. از Skill `dotnet-cqrs-feature` پیروی کن.
 ۴. به Domain و جدول‌های موجود دست نزن؛ ارسال پیامک خارج از محدوده است.
 ۵. هر معیار پذیرش ۱ تا ۴ باید یک unit test داشته باشد؛ `dotnet test` سبز.
 ۶. در پایان خلاصه تغییرات و نگاشت معیار پذیرش به تست بده.

۸.۳ ترفندهای کاربردی

- برای هر تسک یک جلسه/چت تازه؛ context طولانی کیفیت را پایین می‌آورد.
- از عامل بخواهید قبل از کد سؤال‌های روشن‌کننده بپرسد.
- «این کارها موازی قابل انجام‌اند» را صریح بگویید.

- برای بازبینی بی‌طرف: «از subagent ای استفاده کن که گفتگوی قبلی ما را ندیده.»
- قالب خروجی را مشخص کنید (جدول، چک‌لیست، نگاشت AC به تست).

۸.۴ ضدالگوها

ضدالگو	چرا بد است	جایگزین
یک چت بی‌پایان	context پر و آلوده	جلسه جدید برای هر تسک
پذیرش بدون خواندن	مالکیت و مسئولیت با شماست	PR عامل = همان بازبینی
AGENTS.md هزار خطی	هزینه دائمی context، قوانین گم می‌شوند	Rule کوتاه + docs + Skill
«درستش کن» بدون تست	عامل به خودش نمره می‌دهد	معیار تأیید قطعی
Secret در پرامپت یا Rule	نشت اطلاعات	متغیر محیطی + Hook اسکن
اصلاح دستی خطای تکراری	دفعه بعد تکرار می‌شود	افزودن Rule/Skill/تست (روی حلقه)
BRD داخل Rules	context شلوغ و مبهم	docs + خلاصه در Spec

۹. نقشه راه ۳۰/۶۰/۹۰ روزه

بازه	اقدامات	معیار موفقیت
۳۰ روز	AGENTS.md برای ۲ سرویس پایلوت؛ Rule ۲ (معماری، تست)؛ آموزش Plan Mode و subagent؛ ثبت داده پایه	همه اعضای پایلوت حداقل ۵ تسک با Plan Mode
۶۰ روز	Skill ۳ مشترک (CQRS، migration، تست)؛ اولین فیچر کامل با Spec؛ subagent بازبین قبل از PR؛ Hook برای build/test	یک فیچر end-to-end با Spec و معیار پذیرش پوشش داده شده
۹۰ روز	SDD برای همه فیچرهای جدید؛ spec-anchored برای مازول‌های اصلی؛ کتابخانه Skill تیمی؛ بازنگری داده‌ها	مقایسه lead time، نرخ باگ و دوباره‌کاری با پایه

نقش‌ها: Champion (پیشگام و پاسخ‌گوی سؤالات)، Tech Lead (مالک Harness)، Manager (حمایت، بودجه و اندازه‌گیری).

چه چیزی اندازه بگیریم: lead time تا merge، نرخ باگ پس از انتشار، تعداد دور بازبینی PR، درصد فیچرهای دارای Spec، رضایت تیم. یادآوری METR: احساس سرعت با سرعت واقعی یکی نیست.

۱۰. پرسش‌های پرتکرار (FAQ)

۱. آیا AI جای توسعه‌دهندگان را می‌گیرد؟

نقش را تغییر می‌دهد، نه حذف. تمرکز از تایپ کد به طراحی، نوشتن Spec، ساخت Harness و بازبینی می‌رود. مسئولیت و مالکیت کد با انسان است.

۲. قوانین کسب‌وکار Rule هستند یا Skill یا Spec؟

معمولاً هیچ‌کدام از Rule و Skill. جای اصلی‌شان مستندات دامنه است و در هر فیچر به شکل معیار پذیرش در Spec می‌آیند. Rule فقط برای قراردادهای دائمی کدنویسی است.

۳. BRD را داخل AGENTS.md بگذاریم؟

خیر. BRD در `docs/`؛ خلاصه قابل تست مربوط به هر فیچر در Spec. AGENTS.md باید کوتاه بماند.

۴. Spec-first یا spec-anchored؟

با spec-first شروع کنید. برای مازول‌های اصلی و پرتغییر، Spec را بعد از merge نگه دارید و با هر تغییر به‌روز کنید (spec-anchored). Spec-as-source فعلاً آزمایشی است.

۵. چه کسی مالک Spec، Rule و Skill است؟

Spec: PO + Tech Lead. Rule: Tech Lead. Skill: Tech Lead. Doc: PO/تحلیلگر. همه در git و با PR بازبینی می‌شوند.

۶. Spec Kit یا Kiro یا دستی؟

اگر ابزار فعلی‌تان Cursor/Copilot/Claude است، Spec Kit چون روی همان ابزار کار می‌کند. Kiro اگر IDE جدید می‌پذیرید. برای شروع، قالب ساده `WRITE-A-SPEC.md` همین ریپو کافی است.

۷. امنیت کد شرکت چه می‌شود؟

Privacy mode ابزارها را فعال کنید، سیاست نگهداری داده هر ارائه‌دهنده را بررسی کنید، secret را هرگز در پرامپت/Rule نگذارید، دسترسی MCP و ترمینال را محدود کنید و برای اسکن secret از Hook یا CI استفاده کنید.

۸. هزینه چقدر است؟

قیمت پایه ۱۰ تا ۲۰ دلار ماهانه، ولی مصرف عامل با credit محاسبه می‌شود. مدل سنگین را برای Plan و کار سخت، و مدل سریع را برای کارهای تکراری استفاده کنید.

۹. PR های تولید عامل را چطور بازبینی کنیم؟

دقیقاً مثل PR انسانی: همان CI و branch protection. در برابر Spec چک کنید. قبل از PR یک subagent بازبین read-only اجرا شود. PR ها را کوچک نگه دارید.

۱۰. جونیورها چطور یاد بگیرند و وابسته نشوند؟

از عامل «توضیح» بخواهند، Plan را قبل از اجرا بخوانند و نقد کنند، و کد را خودشان review کنند. Skill و Spec راهنمای ساختاریافته برایشان است.

۱۱. با توهم (hallucination) چه کنیم؟

با context درست (Spec، Rule)، حسگرهای قطعی (تست، build) و بازبینی کاهش می‌یابد ولی صفر نمی‌شود. API ها و پکیج‌های پیشنهادی را تأیید کنید.

۱۲. کدام مدل بهتر است؟

مدل‌ها مرتب عوض می‌شوند. مدل قوی برای برنامه‌ریزی و استدلال، مدل سریع برای کارهای ساده. Harness خوب از مدل بهتر مهم‌تر است.

۱۳. چند ابزار هم‌زمان؟ ناسازگار نمی‌شوند؟

با AGENTS.md به‌عنوان منبع اصلی و فایل‌های مخصوص ابزار کوتاه و ارجاع‌دهنده، و Skills در `agents/skills`، همه ابزارها یک قانون را می‌خوانند.

۱۴. Rule چقدر باید طولانی باشد؟

تا حد ممکن کوتاه. اگر بیش از یک صفحه شد، بخش‌های رویه‌ای را به Skill و توضیحات دامنه را به docs منتقل کنید.

۱۵. Hook چه فرقی با Rule دارد؟

Rule توصیه است و مدل ممکن است فراموش کند؛ Hook اسکرپت قطعی است و همیشه اجرا می‌شود. هر چیزی که «حتماً» باید اتفاق بیفتد را Hook کنید.

۱۶. Subagent کی لازم است؟

وقتی کار خواندن زیاد دارد (تحقیق کدبیس)، وقتی ارزیابی بی‌طرف می‌خواهید (بازبینی)، یا وقتی کارهای مستقل موازی دارید.

۱۷. کد legacy شلوغ داریم؛ عامل کمکی می‌کند؟

بله ولی با احتیاط: اول تست‌های characterization با عامل بسازید، بعد refactor. کد بی‌نظم context بدی است؛ Böckeler بر «طراحی کدبیس دوستدار AI» تأکید دارد.

۱۸. از کجا شروع کنیم؟

همین هفته: یک AGENTS.md کوتاه برای یک سرویس، استفاده از Plan Mode برای یک تسک واقعی، و نوشتن یک Spec با قالب این ریپو.

۱۱. واژه‌نامه

اصطلاح	معادل/توضیح فارسی
Agent	عامل: مدل + ابزار + حلقه تصمیم
Agentic Coding	برنامه‌نویسی عامل‌محور
Context Engineering	مهندسی زمینه: گزینش آنچه مدل می‌بیند
Harness	مهار: همه چیز در عامل جز مدل (راهنماها + حسگرها)
Guides / Sensors	راهنماها (قبل از عمل) / حسگرها (بعد از عمل)
Spec	مشخصات فیچر؛ قرارداد «چه بسازیم»
Rule	قانون دائمی پروژه برای عامل
Skill	مهارت: رویه قابل بارگذاری در صورت نیاز
Progressive Disclosure	بارگذاری تدریجی
Acceptance Criteria (AC)	معیار پذیرش
SDD	توسعه مبتنی بر Spec
Spec-first / anchored / as-source	Spec اول / Spec لنگرانداخته / Spec به‌عنوان منبع
Vibe Coding	کدنویسی حسی بدون ساختار
In / On the loop	درون حلقه / روی حلقه
MCP	پروتکل اتصال عامل به ابزارهای بیرونی
Subagent	عامل کمکی با context جدا
Hook	اسکرپت قطعی در رویدادهای عامل
Orchestrator	هماهنگ‌کننده عامل‌ها

حالت برنامه‌ریزی بدون ویرایش	Plan Mode
نسخه کاری جدای git برای کار موازی	Worktree
سند تصمیم معماری	ADR
سند نیازمندی‌های کسب‌وکار	BRD

martinfowler.com / Thoughtworks

Birgitta Böckeler — [Understanding Spec-Driven-Development: Kiro, spec-kit, and Tessel](#)

Birgitta Böckeler — [Context Engineering for Coding Agents](#) (Feb 2026)

Birgitta Böckeler — [Harness engineering for coding agent users](#)

Birgitta Böckeler — [Harness Engineering — first thoughts](#)

Kief Morris — [Humans and Agents in Software Engineering Loops](#)

Thoughtworks Podcast — [What is spec-driven development?](#) (Böckeler & Laura Tacho)

Standards & tool docs

[Agent Skills specification](#) — [agentskills.io](#) · [github.com/agentskills/agentskills](#)

[AGENTS.md](#)

[Cursor — Agent Skills](#) · [Cursor — Rules](#)

[Anthropic — Best practices for Claude Code](#)

[Anthropic — How and when to use subagents in Claude Code](#)

[VS Code — Use Agent Skills](#)

[GitHub Spec Kit](#)

[Kiro](#) · [Tessel](#)

Research & data

[DORA 2025 — State of AI-assisted Software Development](#)

[Google Cloud — Announcing the 2025 DORA Report](#)

[METR — Impact of Early-2025 AI on Experienced OSS Developer Productivity](#)

[InfoWorld — 4 cutting-edge tools for spec-driven development](#) (May 2026)

[Mehul Jain — AI Coding Agents in 2026: A Practical Implementation Guide](#)

[Agentic Coding: A Field Manual for Shipping Software With AI Agents](#)

پیوست: فایل‌های نمونه کامل

همه این فایل‌ها در پوشه `samples/` ریپو هستند. برای خوانایی در PDF، نسخه انگلیسی خلاصه‌شده آمده؛ نسخه ریپو برای AGENTS.md و CLAUDE.md فارسی است.

`samples/AGENTS.md` (English summary)

```
# AGENTS.md - .NET Clean Architecture
```

```
## Project overview
```

- Stack: .NET / C#, Clean Architecture (Domain / Application / Infrastructure / Host)
- Patterns: CQRS with MediatR, FluentValidation, Result pattern
- Tests: xUnit + NSubstitute

```
## Commands
```

```
dotnet build
dotnet test
dotnet format
```

```
## Architecture rules (always)
```

- Domain must not depend on Infrastructure or Host.
- Keep controllers thin; business logic in Handlers/Services.
- Money fields: decimal only – never double/float.
- Never hardcode secrets, tokens, or connection strings.
- For a new feature, read specs/ first; no code without acceptance criteria.

```
## Definition of Done
```

1. Build green
2. Related tests pass
3. Spec acceptance criteria covered
4. No secrets in diff

`samples/CLAUDE.md`

```
# CLAUDE.md
```

```
@AGENTS.md
```

```
## Claude Code specifics
```

- Non-trivial work: Explore → Plan → Code → Commit
- Delegate codebase research to a subagent.
- Run read-only code-reviewer before commit.

`samples/.cursor/rules/testing.mdc`

```
---
description: Unit and integration testing conventions
globs: "**/*Tests*/**/*.cs"
alwaysApply: false
---
```

```
# Testing Rules
- xUnit + NSubstitute; Arrange-Act-Assert.
- Name tests: Method_Scenario_Expected.
- Cover success + at least one failure path.
- No real network/DB in unit tests.
- dotnet test must pass.
```

`samples/.agents/skills/ef-migration/SKILL.md`

```
---
name: ef-migration
description: Create and apply EF Core migrations safely in a .NET project.
  Use when the user asks for a database migration or schema change.
---
```

```
## Steps
1. Confirm entity/config changes are in the Infrastructure project.
2. dotnet ef migrations add <Name> --project <Infra> --startup-project <Host>
3. Review migration for destructive changes.
4. Apply locally only if asked.
5. dotnet build + tests.
## Never
- Never invent connection strings.
- Never drop production data without explicit human approval.
```

`samples/.claude/agents/code-reviewer.md`

```
---
name: code-reviewer
description: Read-only adversarial code reviewer. Use after implementation, before PR.
tools: Read, Grep, Glob, Bash
---
- Do NOT edit files. Focus on NEW issues.
Checklist: spec AC covered? layer boundaries? decimal for money?
secrets absent? meaningful tests? error paths? naming?
Output: Verdict PASS/FAIL + findings (severity, file, why, fix) + spec gaps
```

`samples/specs/001-installment-followup/tasks.md`

- [] T1: DT0 + repository interface for overdue + follow-up
- [] T2: Overdue installments query + unit test (AC1)
- [] T3: Add follow-up command + validator + tests (AC2, AC4)
- [] T4: Return history in details (AC3)
- [] T5: Controller endpoints
- [] T6: dotnet build && dotnet test
- [] T7: Adversarial review before PR

نسخه کامل فارسی spec.md، plan.md و قالب‌های WRITE-A-SPEC / WRITE-A-RULE / WRITE-A-SKILL / DECISION- TREE در ریپو موجود است.